Reason. Plan. Act. Physical AI Workshop

Lab 2.1: Motion Planning part 2

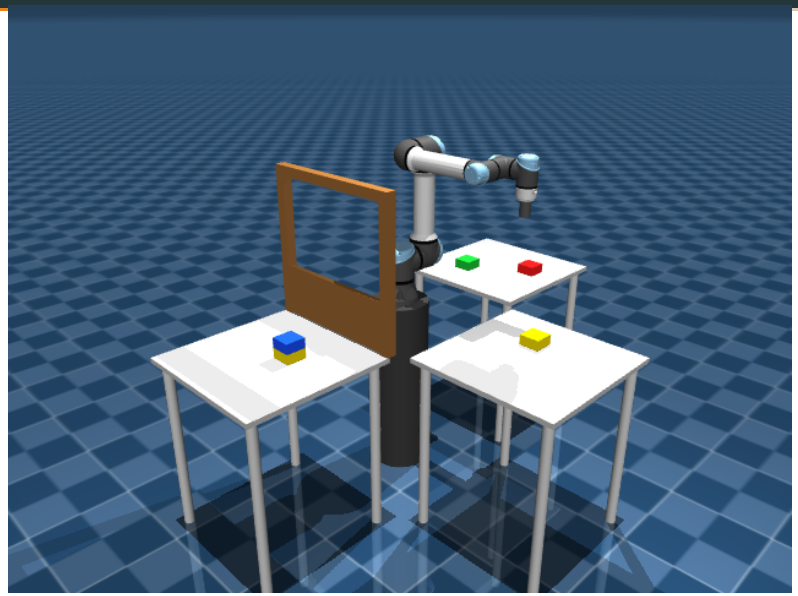
Guy Azran and Sarah Keren

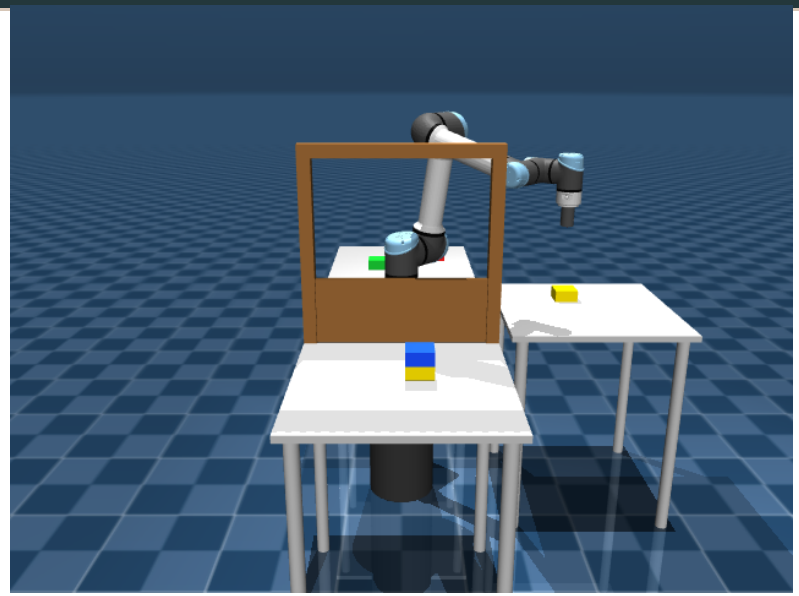
The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



Where we left off

Guy Azran and
Sarah Keren





IK gives a target configuration, and `move_to` goes there in a straight line in configuration space. With a wall in front of `t1`, the line to above `b2` hits the wall 41% of the way, and so do the lines to all eight IK solutions.

1.1 Task planning	**2.1 Motion planning**	3.1 State estimation	4.1 Integrated TAMP
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	4.2 VLAs

Today

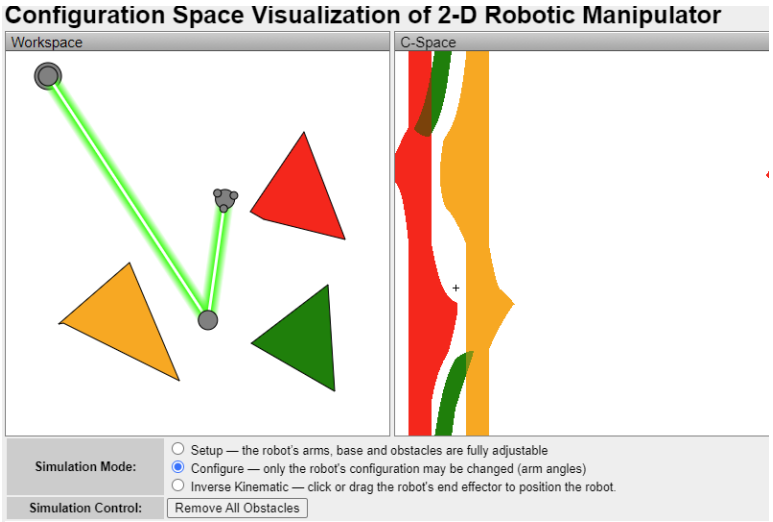
Learn	The motion planning problem, PRM and RRT
Do	Task 1: compare planners in 2D with OMPL
Learn	Planning for the arm: bounds and collision checks
Do	Task 2: the validity checker
Learn	Planning and following a path
Do	Task 3: follow a path
Do	Task 3.1: coarse checks, sparse paths
Learn	Planning while holding a block
Do	Task 4: the block moves with the robot
Learn	Path quality
Do	Task 5: RRT* with growing budgets
Learn	Executing the plan of Lab 1.1
Do	Task 6: the place skill
Do	Bonus: longer plans

The motion planning problem

Guy Azran and Sarah Keren

- $\mathcal{C}$: all configurations $q = (q_1, \ldots, q_6)$
- $\mathcal{C}_{\text{obs}}$: the arm overlaps a table, the wall, a block, or itself
- $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$

Find a path $\tau : [0, 1] \to \mathcal{C}_{\text{free}}$ from q_{start} to q_{goal}. In practice: **waypoints** joined by free straight segments.



Obstacles in the workspace (left) are strange shapes in C-space (right)

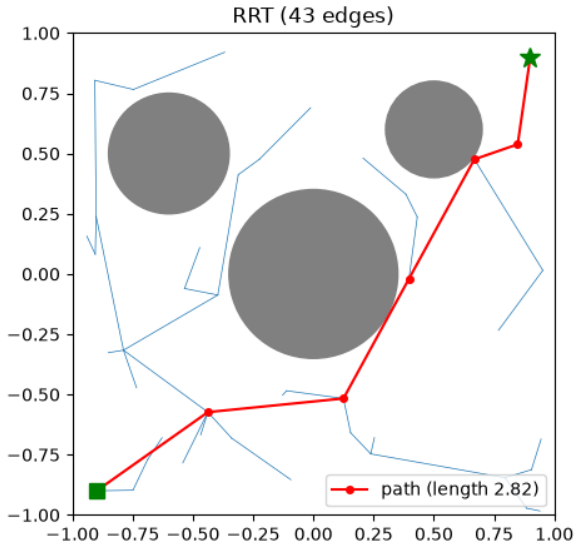
For 6 joints we cannot even draw $\mathcal{C}_{\text{obs}}$. We can only ask, for one q at a time: **is it free?**

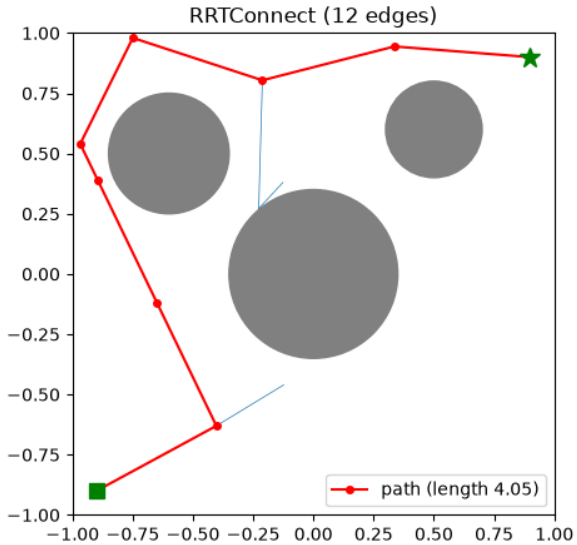
Sampling-based planners

Guy Azran and Sarah Keren

Sample configurations, keep the free ones, connect neighbors by free straight segments. Probabilistically complete: if a path exists, we find it with probability $\to 1$.

- **PRM**: a roadmap, reusable for many queries
- **RRT**: a tree grown from the start; **RRT-Connect**: two trees that meet
- **RRT***, **PRM***: keep rewiring, the path converges to the shortest one





OMPL: the Open Motion Planning Library

OMPL knows nothing about robots. We give it four pieces: a **state space** with bounds, a **validity checker** (it asks *us* whether a state is free), a **problem** (start and goal), and a **planner** with a termination condition.

```
space_info_2d = ob.SpaceInformation(space_2d)
space_info_2d.setStateValidityChecker(is_free_2d)
space_info_2d.setStateValidityCheckingResolution(0.005)
space_info_2d.setup()
```

```
planner_2d = og.RRT(space_info_2d)
planner_2d.setProblemDefinition(problem_2d)
planner_2d.setup()

status = planner_2d.solve(iterations(2000))
```

Reproducible runs: seed right before creating the planner, and budget in iterations, not seconds.

Task 1: compare planners in 2D

Guy Azran and
Sarah Keren

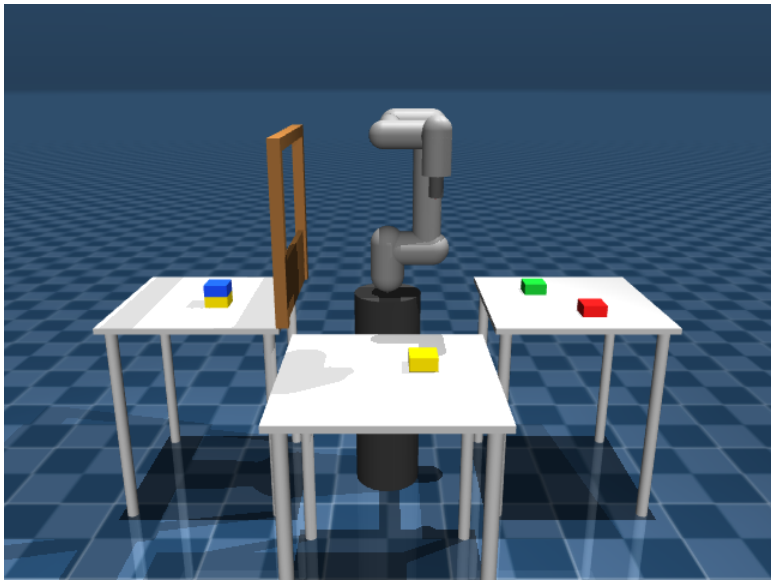
⚒ **Task 1**: Run `plan_2d` with `og.RRT`, `og.RRTConnect` and `og.RRTstar`, with a few different iteration budgets (e.g. 200, 2000, 20000). Compare the size of the graphs, the planning time, and the length of the paths. Which planners stop at the first solution, and which use up the whole budget? Then try `og.PRM` and `og.PRMstar` with a `time_limit` (e.g. 0.01, 0.1 and 1 second). Finally, make the problem harder: move the obstacles (edit `OBSTACLES`) to leave only a narrow passage between the start and the goal, and see how the planners cope.

```
# your code here
```

- Compare graph sizes, planning times and path lengths
- PRM and PRM* only with `time_limit`: their helper thread cannot call back into Python

Planning for the arm

- State space: 6 joint angles in $[-\pi, \pi]$
- Collisions are checked on simple capsules, a bit larger than the links: conservative and cheap
- Any contact of the robot with something else, itself included, is a collision



The robot's collision geometry

A planner asks for tens of thousands of these checks.

Task 2: the validity checker

Guy Azran and
Sarah Keren

⚒ **Task 2**: Write the validity checker `is_valid_configuration`. It receives an OMPL state (which behaves like a list of 6 joint angles, `state[:6]` gives them all), and must return `True` when the robot at that configuration is collision-free. Use `set_configuration` and `robot_in_collision`.

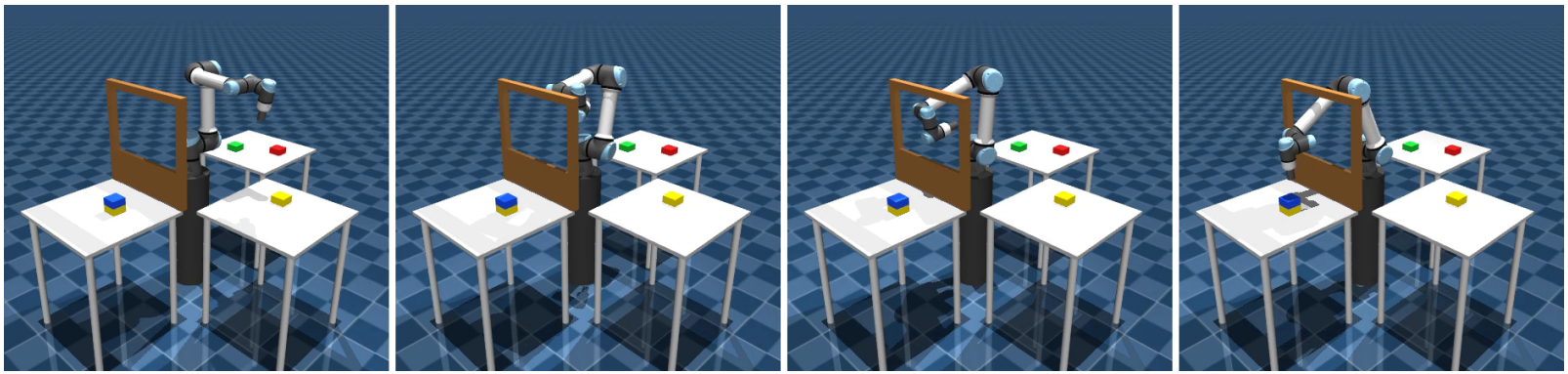
```python
def is_valid_configuration(state):
    """OMPL's validity checker: is the robot collision-free at this configuration?"""
    return True  # your code here
```

- Teleport with `set_configuration`, then ask `robot_in_collision`
- The cell estimates how much of the C-space is free, and the time per check

Planning a path

`plan(q_start, q_goal)` wraps OMPL for the arm:

- saves the state of the simulation and restores it, since the checks teleport the robot
- **simplifies** the path by shortcuts: 20.5 rad $\rightarrow$ 7.7 rad
- **interpolates** it: waypoints at most 0.05 rad apart



From `HOME` to above `b2`, through the window, in 0.07 s of planning.

Task 3: follow a path

Guy Azran and Sarah Keren

⚒ **Task 3**: Complete `follow_path`. For every waypoint except the last, keep applying it as a target (like `move_to` does) until the robot is within `tolerance` of it, and give up if it takes more than `max_steps_per_waypoint` steps. For the last waypoint, use `move_to`. Collect video frames along the way, as `move_to` does.

```
def follow_path(waypoints, tolerance=0.1,
max_steps_per_waypoint=500, render_every=50, camera="overview"):
    """Move the robot along the path. Returns (success,
frames)."""
    frames = []
    step = 0
    for i, target in enumerate(waypoints):
        # your code here
        pass
    return True, frames
```

- Do not stop at every waypoint: move on when within `tolerance`

Task 3.1: coarse checks, sparse paths

Guy Azran and
Sarah Keren

⚒ **Task 3.1**: Plan again with a coarse resolution (`resolution=0.05`) and a sparse path (`spacing=1.0`), and execute it. Does the robot still make it? Look at the video, and count the collisions along the way with `path_in_collision` (use many intermediate steps).

```
# your code here
```

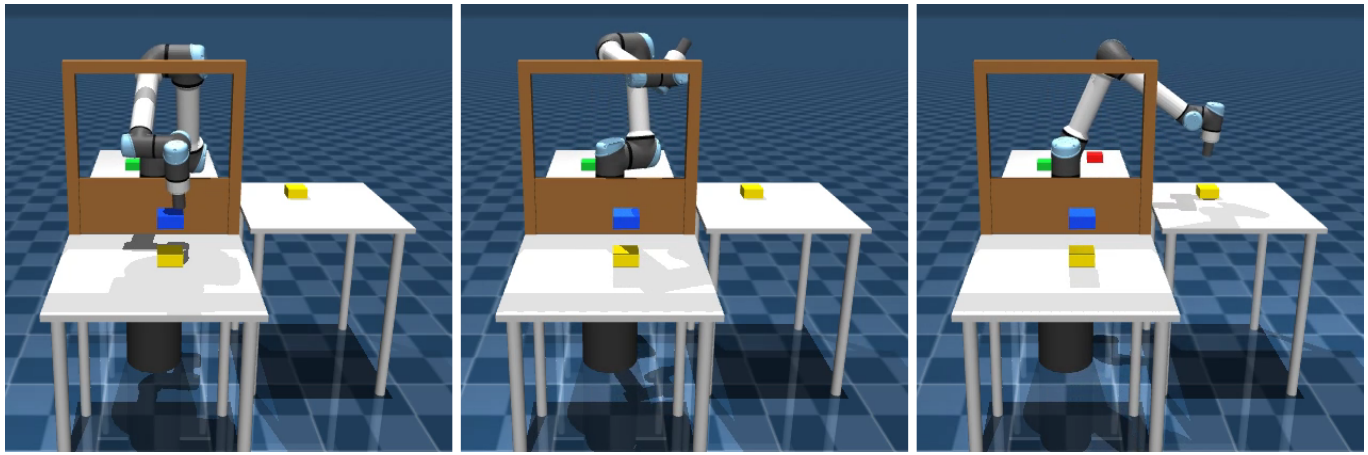
What could still go wrong?

- the planner checks each segment at a finite resolution
- the robot lags behind its targets and cuts corners between waypoints

Planning while holding a block

The held block is part of the robot now, but the planner does not know it:

- `set_configuration` moves only the joints. The weld is a physics constraint, so the block stays behind
- `robot_in_collision` ignores the block's contacts

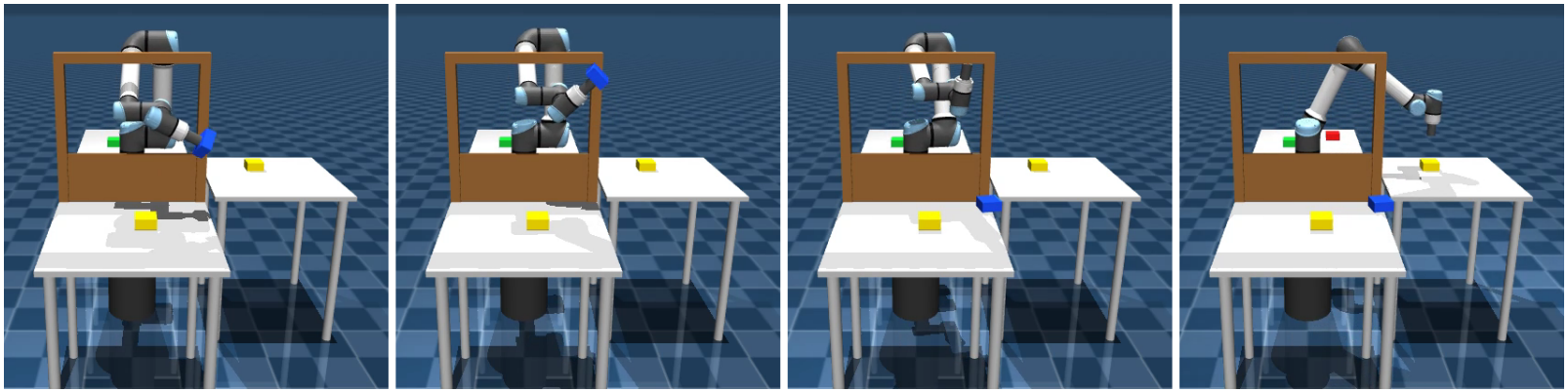


Animating a planned path: the blue block stays behind

7 of 10 paths planned without the block would hit something with it.

What if the robot executes such a path?

A real suction cup has a **holding force**. Ours holds 30 N, thirty times the weight of a block.



- The block hits the window frame, is pushed with 33 N, and the seal breaks
- The arm carries on to its goal: nothing in our pipeline looks at the block after the grasp

Task 4: the block moves with the robot

Guy Azran and
Sarah Keren

⚒ **Task 4**: Complete the first change: compute the block's position and orientation in the world from the cup's pose and the relative pose, in `set_configuration`. This is the same kind of reference frame arithmetic as `world_to_base` in the previous tutorial, in the other direction: $p_{\text{world}} = R_{\text{cup}}\, p_{\text{cup}} + t_{\text{cup}}$. For the orientation, quaternions are composed with `mujoco.mju_mulQuat(result, first, second)`.

```
        position = cup.xpos  # your code here
        quat = cup.xquat  # your code here
```

- `grasp` stored the block's pose in the cup's frame: `relative_position`, `relative_quat`
- `mj_kinematics` gives the cup's pose after the robot moves
- Check: the weld should move the block by about 0 mm

Path quality

From **HOME** to above **b2**, with RRT-Connect:

path	length	waypoints
as found	20.53 rad	411
simplified	7.66 rad	154

- RRT-Connect stops at the first path; the simplifier removes most of the zigzag
- RRT* keeps improving its path for as long as it runs, at a growing cost per iteration

Is the extra planning time worth it? It depends on the application.

Task 5: RRT* with growing budgets

Guy Azran and
Sarah Keren

⚒ **Task 5**: Run RRT* (`og.RRTstar`) from `q_above_b2` to `HOME` with budgets of 500, 1000, 2000, 4000 and 8000 iterations (without the simplifier, to see the planner's own work), and plot the length of the path and the planning time as a function of the budget. Remember that RRT* uses up its whole budget, and that it may return no path at all when the budget is small. Where would you put the budget for a robot that has to move now? And for one that will repeat the same motion a thousand times?

```
budgets = [500, 1000, 2000, 4000, 8000]
# your code here
```

Why backwards? RRT* grows one tree, from the start. The window is a narrow passage that a tree grown from `HOME` rarely finds. A tree grown from above `b2` starts inside it.

Executing the plan of Lab 1.1

Guy Azran and Sarah Keren

`unstack(b2, b1)`, `put-down(b2, t1)`, `pick-up(b3, t2)`, `stack(b3, b1)`

Each action becomes: a target pose, IK, a planned motion to the pre-grasp pose, a straight descent, suction on or off, a straight retreat.

```
top = data.body(block).xpos + [0, 0, BLOCK_HALF_HEIGHT]
success1, frames1 = go_to_pose(top + [0, 0, 0.1])
success2, frames2 = go_to_pose(top + [0, 0, 0.005], straight=True)
grasp()
success3, frames3 = go_to_pose(top + [0, 0, 0.1], straight=True)
```

- `go_to` plans and follows a path
- `go_to_pose` adds IK: the collision-free solution closest to the current configuration

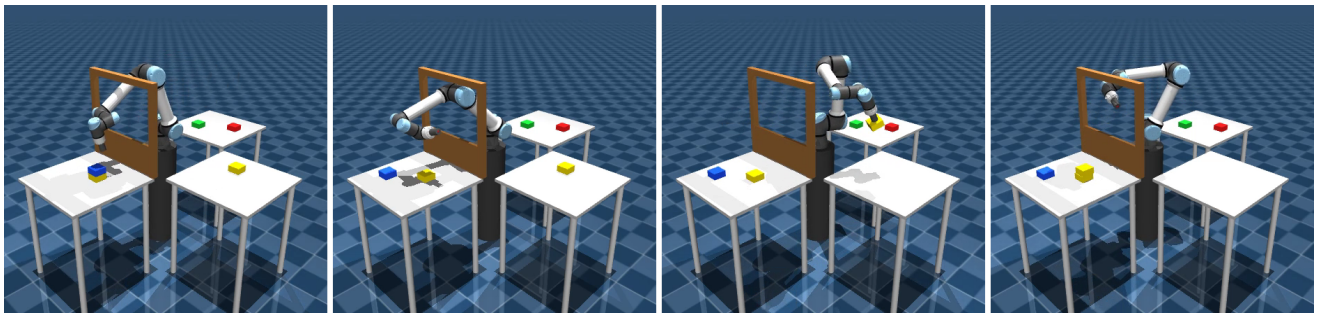
Task 6: the place skill

Guy Azran and Sarah Keren

⚒ **Task 6**: Write `place(target_top)`, the mirror image of `pick_up`: it puts the held block down with its bottom on `target_top`, a point in the world (the top of another block, or a point on a table). Approach a pre-place pose above the target, descend until the block almost rests on the target, release, and retreat. Mind the heights: the held block extends `2 * BLOCK_HALF_HEIGHT` below the cup, and the target is the *bottom* of the block. Then `put_down` and `stack` are one-liners.

```
def place(target_top):
    """put-down / stack: put the held block down with its bottom
on target_top (a point in the world)."""
    return True, []  # your code here
```

The four actions of the plan, through the window:



Bonus task: longer plans

Guy Azran and
Sarah Keren

⚒ **Bonus task**: Execute a longer plan: build a tower of four or five blocks on `t1`, or move the whole tower to `t3`. Try a few seeds. Does every attempt succeed? When one fails, find out where: is it the planner (no path), the IK (no collision-free solution), or the execution (the robot or the block hit something after all)?

```
# your code here
```

When an attempt fails, where does it fail?

- the planner: no path found
- IK: no collision-free solution
- execution: the robot or the block hit something after all

Get to work!

Summary

In this tutorial we

- stated the motion planning problem: a path through the free space, known only through a collision check
- used sampling-based planners in OMPL: PRM, RRT, RRT-Connect, and the optimizing RRT* and PRM*
- made randomized planning reproducible with seeds and iteration budgets
- planned and executed collision-free paths for the arm, through the window
- planned while holding a block, which becomes part of the robot
- traded path quality against planning time
- executed the plan of Lab 1.1, and met the decisions the task planner never made

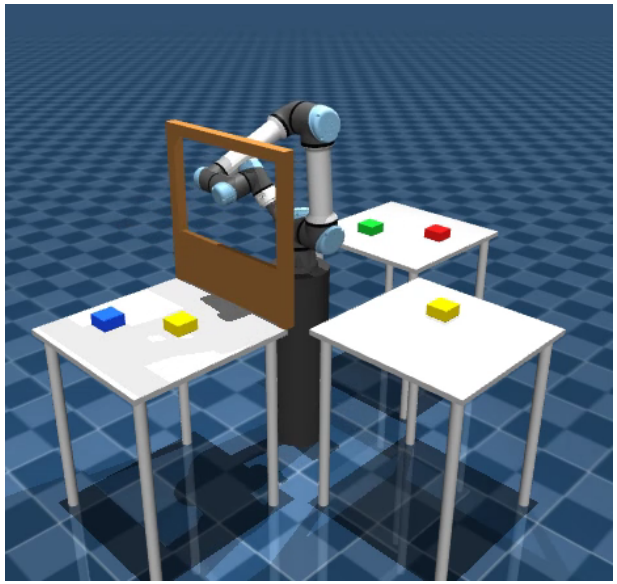
Between the planners, and what comes next

Guy Azran and Sarah Keren

Decisions that the task planner never made:

- *where* on `t1` to put `b2` down
- which IK solution, which of many paths

Every motion ran **open loop**: plan, send the waypoints, assume the robot follows them.



`b2` put down on `t1`

Next:

- **Control**: what is inside the servos, and how to follow paths accurately
- **State estimation**: from observations back to facts
- **TAMP**: the two planners working together